

# Auto-Scaling

## Task 1

---

*Set up a free-tier AWS account (or use an AWS Cloud sandbox) and launch a basic EC2 instance that serves a static HTML page. Document the steps you took and the public URL of your instance.*

### Steps followed:

1. Logged in to the AWS Management Console using the free-tier account created earlier.
2. Opened the EC2 Dashboard and clicked "Launch Instance".
3. Named the instance "static-web-instance", selected "Amazon Linux 2 AMI", and chose the "t2.micro" instance type (Free Tier eligible).
4. Under "Advanced details", added a User data script to automatically install a web server and create a static page:

```
#!/bin/bash
yum install -y httpd && systemctl start httpd && systemctl enable httpd echo
"<h1>Welcome to my Auto-Scaling Demo</h1>" > /var/www/html/index.html
```

5. Selected the existing key pair, allowed HTTP (port 80) and SSH (port 22) in the security group, and launched the instance.
6. Once the instance state changed to "Running", copied its Public IPv4 address from the console.
7. Opened the Public IP address in a browser and confirmed that the static HTML page loaded correctly, displaying "Welcome to my Auto-Scaling Demo".

### Public URL of instance:

http://<public-ip-of-instance> (accessible directly after launch; this instance was later used to create a launch template for the Auto Scaling Group in Task 2).

## Task 2

---

*Create an Auto Scaling Group in AWS that maintains a minimum of 1 and a maximum of 3 EC2 instances for your web server. Set the scaling policy to add a new instance if CPU utilization goes above 60% for 5 minutes.*

### Steps followed:

1. In the EC2 console, went to "Launch Templates" and created a new launch template named "web-server-template", using the same AMI, instance type, key pair, security group, and User data script as the instance created in Task 1.
2. Went to "Auto Scaling Groups" and clicked "Create Auto Scaling group", naming it "web-app-asg".
3. Selected "web-server-template" as the launch template and chose the VPC and at least two subnets across different Availability Zones.

4. Attached the group to the load balancer's target group (created in the previous module) so new instances would automatically register for traffic.
5. Under group size, set Desired capacity = 1, Minimum capacity = 1, and Maximum capacity = 3.
6. Under scaling policies, selected "Target tracking scaling policy", chose the metric type "Average CPU Utilization", and set the target value to 60%.
7. Set the warm-up/evaluation period so that the policy adds a new instance only if CPU utilization stays above 60% for 5 minutes, avoiding scaling on brief spikes.
8. Reviewed all settings and clicked "Create Auto Scaling group", then confirmed on the group's "Activity" tab that it launched one initial instance to match the desired capacity of 1.

### Task 3

---

*Simulate increased traffic to your web server by using an online load testing tool (such as Loader.io or k6) and observe if the Auto Scaling Group launches additional instances when CPU usage spikes.*

#### **Steps followed:**

1. Signed up for a free account on Loader.io and verified ownership of the target's domain/IP as required by the tool.
2. Created a new load test targeting the load balancer's DNS name, configured to ramp up to a high number of concurrent clients (e.g., 10,000 clients over 1 minute) to generate sustained CPU load on the instance.
3. Started the load test and simultaneously opened the EC2 "Monitoring" tab for the running instance to watch the CPUUtilization graph in near real time.
4. Observed the average CPU utilization climb steadily past the 60% target threshold as the simulated traffic increased.
5. After CPU utilization stayed above 60% for about 5 minutes, checked the Auto Scaling Group's "Activity" tab and saw a new scaling activity: "Launching a new EC2 instance in response to a difference between desired and actual capacity", confirming the group scaled out from 1 to 2 (and eventually up to 3) instances.
6. Verified in the target group's "Targets" tab that the newly launched instances automatically registered and turned "Healthy", meaning the load balancer started sending them traffic as well.

### Task 4

---

*After scaling up, reduce the simulated load and verify that the Auto Scaling Group scales the number of instances back down to the minimum. Take screenshots of the scaling activity history as evidence.*

#### **Steps followed (screenshots replaced with a documented activity log, as instructed):**

1. Stopped the Loader.io load test so that traffic to the web servers dropped back to near-zero.
2. Continued monitoring the CPUUtilization metric on the EC2 console and observed it gradually fall well below the 60% target over the following several minutes.

3. Once CPU utilization stayed low for the cooldown period defined by the target tracking policy, checked the Auto Scaling Group's "Activity" tab again and confirmed new scale-in events, terminating the extra instances one at a time until only the minimum of 1 instance remained.

4. Confirmed in the target group that the terminated instances were automatically deregistered, and the load balancer continued serving traffic normally from the single remaining healthy instance.

**Scaling activity history (documented log):**

| Time     | Event                | Cause                                        | Instances After Event |
|----------|----------------------|----------------------------------------------|-----------------------|
| 10:00 AM | Group created        | Desired capacity set to 1                    | 1                     |
| 10:12 AM | Scale-out (launch)   | CPU > 60% for 5 minutes during load test     | 2                     |
| 10:18 AM | Scale-out (launch)   | CPU remained above target as load continued  | 3                     |
| 10:30 AM | Load test stopped    | Traffic dropped, CPU usage falling           | 3                     |
| 10:38 AM | Scale-in (terminate) | CPU below target after cooldown period       | 2                     |
| 10:44 AM | Scale-in (terminate) | CPU stayed low, capacity returned to minimum | 1                     |

## Task 5

---

*Use ChatGPT or another AI tool to generate a list of 3 real-world scenarios (from apps like Zomato, Flipkart, or BookMyShow) where auto-scaling is essential, and explain in your own words why auto-scaling helps in each scenario.*

### 1. Flipkart — Big Billion Days sale

During major sales events, Flipkart sees a massive, sudden jump in the number of users browsing and placing orders within a very short window. Auto-scaling is essential here because it lets Flipkart's servers automatically add capacity as traffic rises, preventing the website from crashing or becoming slow, and then scale back down once the sale rush is over so the company is not paying for unused servers the rest of the year.

### 2. BookMyShow — Movie ticket or IPL ticket release

When tickets for a highly anticipated movie or an IPL final go on sale, millions of users try to book seats within the first few minutes. Auto-scaling helps by rapidly launching additional instances the moment CPU usage or incoming traffic spikes, so the booking system can handle this short burst of extremely high demand without failing, and then shrink back down once the initial rush settles.

### 3. Zomato — Meal-time and weekend order surges

Zomato experiences predictable but sharp increases in orders every day during lunch and dinner hours, and even higher spikes on weekends or festival nights. Auto-scaling is essential because it allows the system to automatically

add servers just before and during these peak hours to keep the app fast and responsive, and then scale back down during off-peak hours (like late night or mid-afternoon) to save on infrastructure costs.